

Chaos Validation Experiment for Anomaly Detection for EC2 CPU

Spikes:

Chaos Experiment: Hypothesis of alert being triggered when CPU spike in system.

Launching an EC2 instance and installing CloudWatch Agent

Configuring CloudWatch metrics for CPU utilisation

Enabling Anomaly Detection for high CPU spikes

Simulating CPU stress using a load generator

Observing and analysing anomalies in the CloudWatch Dashboard

Send CPU Spike Alert to Email

.....

Step 1: Install Terraform and AWS CLI on Ubuntu

Ensure you have **Terraform** and **AWS CLI** installed and configured.

1.1 Install Terraform

```
sudo apt update && sudo apt install -y wget unzip
```

```
wget
```

```
https://releases.hashicorp.com/terraform/1.7.0/terraform\_1.7.0\_linux\_amd64.zip
```

```
unzip terraform_1.7.0_linux_amd64.zip
```

```
sudo mv terraform /usr/local/bin/
```

```
terraform -v # Verify installation
```

1.2 Install AWS CLI

```
https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
```

```
sudo apt install -y awscli  
aws --version  
aws configure
```

Provide:

- **AWS Access Key**
- **AWS Secret Key**
- **Default region (e.g., us-east-1)**
- **Output format (json)**

Step 2: Create Terraform Configuration for EC2, CloudWatch and Alert

2.1 Create a New Terraform Project

```
mkdir ec2-anomaly-detection  
cd ec2-anomaly-detection
```

2.2 Create the Terraform File (main.tf)

```
provider "aws" {  
  region = "us-east-2"  
}
```

```
# EC2 Instance with CloudWatch Agent and Stress Tool  
resource "aws_instance" "ec2_instance" {  
  ami          = "ami-0cb91c7de36eed2cb" # Replace with  
latest Amazon Linux AMI  
  instance_type = "t2.micro"  
  key_name      = "test1" # Replace with your key pair
```

```
tags = {  
  Name = "Anomaly-Detection-EC2"  
}
```

2.3 Initialize and Deploy Terraform

```
terraform init  
terraform plan  
terraform validate  
terraform apply -auto-approve
```

Install and Configure CloudWatch Agent

SSH into the EC2 Instance

```
chmod 400 "key.pem"  
ssh -i your-key.pem ubuntu@<EC2-Public-IP>
```

Install CloudWatch Agent

Run the following commands on your **Ubuntu EC2 instance**:

```
wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
```

```
sudo dpkg -i amazon-cloudwatch-agent.deb
```

Configure CloudWatch Agent

Run the configuration wizard:

```
sudo amazon-cloudwatch-agent-config-wizard
```

Start and Enable the Agent

```
sudo systemctl start amazon-cloudwatch-agent  
sudo systemctl enable amazon-cloudwatch-agent
```

Verify CloudWatch Agent is Running

```
sudo systemctl status amazon-cloudwatch-agent
```

Step 3: Enable Anomaly Detection for CPU Spikes

Modify main.tf to include CloudWatch Anomaly Detection.

3.1 Add CloudWatch Anomaly Detection Rule

Anomaly Detection for CPU Spikes

```
resource "aws_cloudwatch_metric_alarm" "cpu_anomaly" {  
  alarm_name      = "CPU_Anomaly_Detection"  
  comparison_operator = "GreaterThanUpperThreshold"  
  evaluation_periods = 2  
  threshold_metric_id = "ad1"  
  alarm_description = "Triggers when CPU spikes beyond  
anomaly threshold"  
  actions_enabled   = true  
  alarm_actions      = [aws_sns_topic.cpu_alerts.arn] #  
Sends email alert  
  
  metric_query {  
    id      = "m1"  
    return_data = true  
    metric {
```

```

    metric_name = "CPUUtilization"
    namespace   = "AWS/EC2"
    period      = 60
    stat        = "Average"
    dimensions = {
        InstanceId = aws_instance.ec2_instance.id
    }
}

metric_query {
    id          = "ad1"
    return_data = true
    expression  = "ANOMALY_DETECTION_BAND(m1, 2)"
}

# **CPU Utilization > 75% Alarm (Sends Email)**
resource "aws_cloudwatch_metric_alarm" "cpu_threshold" {
    alarm_name          = "CPU_Threshold_Exceeded"
    comparison_operator = "GreaterThanThreshold"
    evaluation_periods  = 2
    threshold            = 75.0 # **Triggers when CPU > 75%**
    alarm_description   = "Triggers when CPU utilization
exceeds 75%"
    actions_enabled     = true
    alarm_actions       = [aws_sns_topic.cpu_alerts.arn] #
Sends email

    metric_name = "CPUUtilization"
    namespace   = "AWS/EC2"
    period      = 60
    statistic   = "Average"
    dimensions = {
        InstanceId = aws_instance.ec2_instance.id
    }
}

```

}

3.2 Apply the Terraform Changes

```
terraform apply -auto-approve
```

Step 4: Simulate CPU Load

4.1 SSH into the EC2 Instance

```
ssh -i your-key.pem ubuntu@<EC2-Public-IP>
```

4.2 Run CPU Stress Test

Install stress on Ubuntu

Run the following command to install the stress tool:

```
sudo apt update && sudo apt install -y stress
```

Verify Installation

Check if stress is installed:

```
stress --version
```

To **fully load your CPU (100% usage)**, use the following stress command:

Run Full CPU Stress Test

Simulate a CPU spike on EC2:

```
sudo stress --cpu 2 --timeout 300
```

Or

```
sudo stress --cpu $(nproc) --timeout 600
```

Step 5: Cleanup (Destroy Resources)

If you no longer need the setup, run:

```
terraform destroy -auto-approve
```

.....

FULL Config Snippet

```
provider "aws" {  
  region = "us-east-2"  
}
```

```
#  SNS Topic for Email Alerts  
resource "aws_sns_topic" "cpu_alerts" {  
  name = "cpu-anomaly-alerts"  
}
```

```
resource "aws_sns_topic_subscription" "email_subscription" {  
  topic_arn = aws_sns_topic.cpu_alerts.arn  
  protocol = "email"
```



```
    endpoint = "ashish.9.sharma@niit.com" #  Replace with  
your email  
}
```

```
#  EC2 Instance with CloudWatch Agent and Stress Tool  
resource "aws_instance" "ec2_instance" {  
  ami          = "ami-0cb91c7de36eed2cb" # Replace with  
latest Amazon Linux AMI  
  instance_type = "t2.micro"  
  key_name      = "test1" # Replace with your key pair
```

```
  user_data = <<-EOF  
    #!/bin/bash  
    sudo apt update -y  
    sudo apt install -y stress
```

```
    #  Install CloudWatch Agent  
    wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb  
    sudo dpkg -i amazon-cloudwatch-agent.deb  
    sudo systemctl enable amazon-cloudwatch-agent  
    sudo systemctl start amazon-cloudwatch-agent  
    EOF
```

```
  tags = {  
    Name = "Anomaly-Detection-EC2"  
  }  
}
```

```
#  Anomaly Detection for CPU Spikes  
resource "aws_cloudwatch_metric_alarm" "cpu_anomaly" {  
  alarm_name      = "CPU_Anomaly_Detection"  
  comparison_operator = "GreaterThanUpperThreshold"  
  evaluation_periods = 2  
  threshold_metric_id = "ad1"
```



```

    alarm_description = "Triggers when CPU spikes beyond
anomaly threshold"
    actions_enabled   = true
    alarm_actions     = [aws_sns_topic.cpu_alerts.arn] # 💎
Sends email alert

```

```

metric_query {
  id      = "m1"
  return_data = true
  metric {
    metric_name = "CPUUtilization"
    namespace   = "AWS/EC2"
    period      = 60
    stat        = "Average"
    dimensions = {
      InstanceId = aws_instance.ec2_instance.id
    }
  }
}

```

```

metric_query {
  id      = "ad1"
  return_data = true
  expression = "ANOMALY_DETECTION_BAND(m1, 2)"
}

```

```

# ✅ **CPU Utilization > 75% Alarm (Sends Email)**
resource "aws_cloudwatch_metric_alarm" "cpu_threshold" {
  alarm_name          = "CPU_Threshold_Exceeded"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 2
  threshold            = 75.0 # 🐞 **Triggers when CPU > 75%**
  alarm_description   = "Triggers when CPU utilization
exceeds 75%"

```

```
actions_enabled    = true
alarm_actions      = [aws_sns_topic.cpu_alerts.arn] # 
```

Sends email

```
metric_name = "CPUUtilization"
namespace   = "AWS/EC2"
period      = 60
statistic    = "Average"
dimensions = {
  InstanceId = aws_instance.ec2_instance.id
}
}
```